Revised Implementation Plan: Avenir Payment Foundations

Oct 15, 2026 · @Shantha

0. Objective, boundary and what this plan supersedes

Objective, as you set it on 15 October: a reliable, professionally structured Avenir system, so that accurate data can be entered again from a starting date you will confirm. The software is the work; historical reconciliation, old duplicate amounts and corrections to old accounting records are set aside for this phase and must not block it. The existing data is development material: it reproduces faults and shows workflows, and nothing in it (amounts, statuses, behaviour) is taken as correct. Your monthly bank checks found no duplicate bank payments; that stands as recorded. **[Your instruction]**

What this plan supersedes. The consolidated plan and the Final Next-Step Package of 15 October remain the record of the read-only investigation and of Ravi's pack for voucher 7134 (prepared, not sent). Their Stream H (historical reconciliation) is parked, not cancelled. Their Stream S (software faults) is replaced by sections 3–5 here, widened from the authorisation guard to the whole payment workflow. The structural compatibility register's open items stay open where this plan says so. **[Proposal]**

Boundary, applied throughout. Work is on the isolated localhost copy only (section 2). The staff's live system is not accessed or changed. The development copy at C:\wamp64\www\avenir and its database are not changed by this plan; they are read for evidence. No existing record is deleted, reset or overwritten: test records are added, labelled TEST, and listed. Staff assignments and permissions are yours; the plan names what each change needs, never grants it. Approvals: each build, each install on the test copy beyond this one, each write test set and the clean database each need your word; this plan asks for them in section 7.

Labels. [Observed] with its source; [Deduced]; [Proposal]; [Assumption] where not verified; [Blocker] where something cannot proceed without you or IT.

1. Already proven (evidence on record; not to be repeated)

Proven	Evidence	Source
The code has four authorisation paths (V1 Voucher Print / Bank Payment Check, V2/V3 the desk's Paid which calls V1 internally, V4 bulk hotel accept, V5 advance transfer); only the desk refuses a second authorisation; V1 refuses only a voucher marked rej	code read, file:line references	follow-up report section 6 (F-1 to F-7)
The ledger tables account_runing and account_runing_2024 are MyISAM with no primary key and a non-unique ID; voucher has a unique index on Vnum; v_inv has no index on V_tkn	M2, 15 Oct	evidence folder followup-run\06
A second authorisation of one voucher posts its ledger again: voucher 7134 carries two identical journals (26061, 26075; 13 Jul 2026); every other sup = 0 authorised voucher carries one	G1c, G2r, G3, G4	followup-run\08–14; Final package section 12
Request statuses can be changed from any state by seven routes without a status check or lock (approval, bulk approval, create_bulk_voucher, save_payment, completion, lunch_pay, cancellation); bulk rejection sets a bulk-type request to 6 (approved); a bulk lunch approval sets every other request of the hotel account to 6	code read	A2 section 4a; R0 R14; r3.5 12b
Dashboards derive “paid” from tour flags that disagree with stored status (266 at status 4 show flag 1; 21 at status 2 show flag 0)	R0 data	R0 R16

Proven	Evidence	Source
The voucher-number counter is read-then-written without a lock (invoice_no), so two writers can take one number; lines are written before the voucher	code read	follow-up report D9; InvoiceNumberModel
Bank Payment Check's mark_finalized checks neither Aflag nor finalized nor separation	code read	follow-up report D11
Driver advances: three creation routes check nothing against each other; rejecting a Transport Advance request resets every request row of the driver on the tour, paid ones included; the top-up repays posted expenses	code read	F-2, F-3, D3
No creation route has a duplicate check; CSRF is sent by pages but checked nowhere	code read	D8; F-7
The isolated test copy avenir_rv exists, forces every connection to avenir_restore_verify_20261011, shows a red bar, has its own cookies and TEST logins	README-avenir_rv.txt; Database.php override; T-rv read (15 Oct 12:58 UTC)	section 2
MySQL 8.4.7 applies MAX_EXECUTION_TIME hints; performance_schema is on	H0, H1 × 10	followup-run\02–21

Not proven, and treated as unverified: that any existing posting rule is correct; that the current status model (0, 1, 2, 3, 4, 5, 6) is complete or right; that the existing audit (tpd_audit, from 11 October) is tamper-proof; that the application's reports agree with the underlying records. Each of these is tested, not assumed, in section 5.

2. The isolated workspace and the backup and restore arrangements, with the unresolved restrictions named

The isolated workspace: C:\wamp64\www\avenir_rv [Observed, 15 Oct 12:5x UTC]

Item	Fact	How confirmed
Code	a copy of the development code of 11 October with the later kits; 876 PHP files in app, 1,726 files in app\ and public\	file count in the linked folder
Address	http://localhost/avenir_rv/ (this laptop only); every page carries the red bar “ISOLATED TEST COPY (avenir_rv) … dummy records only — not the development system”	App.php baseURL; the page itself
Database	avenir_restore_verify_20261011 only: app\Config\Database.php forces the groups default, secondary and secondDatabase to it in code, so .env cannot point it elsewhere	Database.php lines 115–135; T-rv: DATABASE() = avenir_restore_verify_20261011, MySQL 8.4.7, port 3306
Sessions	its own cookies (avenir_rv_session, avenir_rv_csrf); signing in here never signs anyone out of development	README-avenir_rv.txt
External services	no live keys in .env (no SMS, WhatsApp, AI or e-mail credentials)	README; .env holds CI_ENVIRONMENT only
Logins	TEST users: TestTourRes, TestHotelRes, TestTransport, TestFinApprover, TestBankAuth, TestAccounts, TestOperations (user ids 116–122) plus test, test2	user_acc_tbl, ids and names only
Data	the development data of 11 October (voucher 7,975; v_inv 8,178; account_runing 11,390 before today's TEST rows) with dummy records: tour 990001 / AH TEST-9901, drivers 6-9-11-99901/99902, ledger ids 99990001–02	counts 15 Oct; README

Backups taken on 15 October before any change, and how each is verified [Observed]

Backup	Where	Verification
Files: app\ of the test copy (1,001 files)	C:\wamp64\www\avenir_rv\writable\app_backups\app-before-payguard-20261015-125912.tar.gz (SHA-256 b291e70c…) with app-before-payguard-20261015-125912.sha256 (1,001 lines)	the archive lists 1,001 files; the manifest has 1,001 entries
Tables the fix and tests can write: voucher, v_inv, account_runing, invoice_no, tour_quotations, consignee, driver_allocation, tpd_audit	copies inside the test database, zz_bak_20261015_1259_<table>	row counts equal (7,975 / 8,178 / 11,390 / 1 / 11,089 / 1,869 / 630 / 9,719) and CHECKSUM TABLE EXTENDED equal for every pair
Restore	files: unpack the archive over app\ and compare with the manifest; tables: `RENAME TABLE` the copy back over the table, or `DELETE` the TEST rows by their listed ids (section 4)	both rehearsable on the test copy before any use

Reusing existing permissions. The folder C:\wamp64\www\avenir_rv is already connected to this session (read/write), the test database is reached through phpMyAdmin with your sign-in, and the TEST logins exist: nothing new was granted and nothing is asked for. Deletion in the connected folder is off (files I replace are moved to writable_to_delete, never deleted). **[Observed]**

Unresolved restrictions, stated precisely [Blocker where marked]

1. **Standing of avenir_rv under the structural restriction.** It lies inside C:\wamp64\www but outside C:\wamp64\www\avenir. Your instruction of 15 October to “prepare and test the fix on the isolated localhost test copy” is read as naming it; if the restriction excludes it, say so and the work stops there. *[Assumption, stated]*
2. **Database backup method.** The table copies above live inside the same MySQL server and database as the test copy. A dump to a file would need either MySQL's own file output (secure_file_priv not yet read: S1 of the Final package) or a client on this laptop that the session's shell cannot reach (it cannot connect to your localhost MySQL). A copy outside the server is therefore not on record. *[Blocker only for a disaster that loses the MySQL data directory; not for the tests]*

3. **Verified restore on a separate server** (r3.5 17b) is not possible inside the structure without a second MySQL instance (option T2); for the test copy it is not required, for development it remains the open item U-5 of the register.
4. **Two different users at the same moment** need two signed-in browsers; the TEST passwords are not in any project fixture, so only you can sign them in (section 4 records what was possible).
5. **No PHP runs in the session's shell on this laptop**, so syntax is checked by a one-off page on the test copy (used on 15 Oct and moved out of public\ afterwards) and behaviour by the application itself; PHPUnit suites in tests\ cannot be run from here. *[Limitation]*

3. What needs repair, stage by stage

Order of work: each item is built on the test copy, tested against section 5, packaged with before-copies and checksums, and installed on development only with your approval after a verified backup. Retain-or-refactor is decided per item from the evidence: where the existing code is sound apart from a missing check, the check is added (R-1, R-2, R-4); where the mechanism itself is unsafe on MyISAM, the step is rebuilt around a lock, a claim and an undo (R-1, R-5); a rewrite of the whole chain is not assumed. **[Proposal]**

#	Stage	Fault on record	Repair	Status
R-1	Ledger posting at authorisation (V1, desk Paid via V1, V4, V5)	a second authorisation posts again (D1); V4 posts k times (D2); V5 posts with no checks (D10); a failure part-way leaves rows with no undo (MyISAM)	**the authorisation guard**: named lock per voucher, re-read under the lock, InnoDB claim row, undo of every row written on failure, audit of every outcome; V4 withdrawn; V5 guarded with an operation key	**built on the test copy 15 Oct; tests in section 4**

#	Stage	Fault on record	Repair	Status
R-2	Request status changes (P1, P2, P3, P4, P5, P6, P7, Q1, Q2)	no status check or lock on seven routes; bulk rejection → 6; bulk lunch approval flips other requests; issue_voucher can leave a request at 1	one status engine: every change goes through a compare-and-set (`UPDATE … WHERE status = <expected>` and affected rows = 1) under a per-request named lock; allowed transitions written once (1→4/6, 1→5, 1→3, 4/6→2 with a finalized voucher); bulk actions act only on the selected rows; a transition row per change	next after R-1
R-3	Paid items returning to queues	Q1 cancels any status; transport rejection resets paid rows; lunch approval flips completed requests; dashboards read tour flags	R-2′s transitions make these impossible; the transport reset and the lunch side-effect are removed; panels read stored status, flags become display-only	with R-2
R-4	Voucher creation and numbering	read-then-write counter; lines written before the voucher; no duplicate key on creation routes (D8, D9)	number taken under a named lock with `UPDATE invoice_no SET V_Num = LAST_INSERT_ID(V_Num + 1)` ; voucher row written before its lines, lines carry the number; a business key per creation route (request id, bill id, settlement id) in an InnoDB key table; double submit refused	after R-2

#	Stage	Fault on record	Repair	Status
R-5	Bank-payment recording (Bank Payment Check mark_finalized; desk Paid)	finalises an unposted, cancelled or already finalized voucher; overwrites a bank date (D11); a desk Paid failure after finalized leaves charges unposted (F-4)	state checks (Aflag = 1, finalized = 0) under the voucher lock; bank date written once; the desk′s post-finalize steps made resumable with their own claim	after R-4
R-6	Completion (P6, P7)	completes any request regardless of status or payment	completion only from 4/6 with a finalized voucher whose amount covers the request; part of R-2′s engine	with R-2
R-7	Driver advances and expenses	three uncoordinated creation routes; top-up repays posted expenses (D3); two accepts post twice (D5); settlement double charge (D7)	one creation path with the R-4 key; accept under a lock, one posting per expense; top-up limited to the unpaid balance; the settlement posting rule referred to the accountant before change	after R-4
R-8	Permissions and audit	routes pass on menu paths alone; CSRF unchecked; tpd_audit has no protection against update or delete; legacy screens keep no audit	action-level checks per payment route (who may approve, authorise, finalize, complete), the CSRF filter on payment routes, append-only triggers on the audit table, audit calls in every payment route (R-1 already writes them)	alongside R-2 to R-6

#	Stage	Fault on record	Repair	Status
R-9	Reporting	reports and dashboards computed from flags and from ledger rows that may be duplicated	each report rebuilt from stored status and from one posting per voucher, with a reconciliation check (report total = sum of records) in the test suite	last
R-10	Storage engine	eight payment tables MyISAM: no transaction, no rollback, duplicate journal ids possible	two paths, your decision: keep MyISAM and rely on lock + claim + undo (R-1′s pattern) for every writer; or convert the eight tables to InnoDB (r3 section 9 rehearsal) and add real transactions. Either way: a unique key on the journal id is added so two journals can never share an id	decision needed (section 7)

Integrity demonstration required for every item (section 5): a failure injected after the first write must leave the tables exactly as before the attempt (counts and checksums equal), the status unchanged, and an audit row saying what was undone.

4. Repair item 1, built on the test copy: the authorisation guard (tf-payguard-r1)

4a. What was changed, where, and why it holds on MyISAM

Target confirmed before any change (section 2): C:\wamp64\www\avenir_rv, database avenir_restore_verify_20261011, MySQL 8.4.7, PHP 8.1.33. Backups taken and verified first (section 2). The live staff system and the development copy were not touched. **[Observed, 15 Oct]**

File (under C:\wamp64\www\avenir_rv\app)	Change	Status
Libraries\PayGuard\AuthGuard.php	**new**, 255 lines: claim / complete / fail, snapshot and undo, audit; one named lock per voucher number (GET_LOCK, 10 s wait); re-read of the voucher and of both ledger tables under the lock; an InnoDB claim table pg_auth_claim with the voucher number as primary key; compensation that deletes the rows written in the attempt by their journal ids and restores voucher status, line status, night flags and transport markers from a snapshot taken under the lock	new, compiles
Config\PayGuard.php	**new**: `testHooks` flag, true only on the test copy (release value false); it enables two request fields, pg_delay_ms and pg_fail, used by the tests below and ignored elsewhere	new, compiles
Controllers\Accounts\VoucherPaymentController.php (V1)	`update()`: the claim is taken before any posting when status = 1 and refused with a message (HTTP 409) if the voucher is already authorised, rejected, cancelled, claimed, or already has voucher-payment rows in either ledger table; the whole posting runs in try/catch and any failure calls `AuthGuard::fail()`, which undoes the attempt and answers 500 with what was undone; `acc_array()` records each journal id it writes and throws instead of returning “error” silently; the sup = 2 clearing journal now posts only when authorising (status 1), never on a rejection (the F2 defect); the voucher-number counter is consumed only after a successful claim	edited, compiles
Controllers\Settings\ManualUpdateController.php (V4)	`hotel_voucher_accept()` withdrawn: answers 403 with a message and writes an audit row; the former body kept as a private, unrouted method for reference	edited, compiles

File (under C:\wamp64\www\avenir_rv\app)	Change	Status
Controllers\NewDashboard\DriverAdvancePaymentController.php (V5)	`transfer_amount()` rebuilt around the same pattern: lock on the original voucher; facts read from the database (the original must be authorised and not rejected; the amount must be above zero and within the voucher's amount; the drivers must differ; a transport-advance ledger entry must exist); an operation key per form (table pg_op, primary key) so a repeated submission is refused; undo of the ledger pair, the amount change, the new line and the new voucher if a later step fails; audit of every outcome	edited, compiles
Views\pages\accounts\voucher_print.php	the page now shows the guard's message in full for 409 and 500 instead of "Error occurred"	edited
Views\pages\dashboard2\transport\transport_advance.php	the transfer form sends an operation key generated when the form opens	edited

Routes covered. V1 accounts/voucher_auth/update (Voucher Print and Bank Payment Check post to it); the Tour Payment Desk's Paid and bulk Paid, which call V1's `update()` internally (TourPaymentDeskController :1458) and therefore pass through the same claim; V4 settings/manual_update/hotel_voucher_accept (withdrawn); V5 dashboard2/transport/driver_advance/transfer_amount. No other current code sets Aflag = 1 (follow-up report section 6). **[Observed: code]**

Why it holds without transactions. The payment tables are MyISAM, so nothing here relies on rollback. Two requests for one voucher are serialised by the named lock whatever their route; the decision to post is taken from a re-read under that lock, not from what the page sent; the claim row's primary key makes a second claim fail in InnoDB even if a lock were lost; and a failure after rows were written is undone by deleting exactly those rows (known by journal id) and restoring the snapshot, so the tables return to their state before the attempt. A disabled button is not relied on anywhere. **[Deduced from the design; proven by the tests in 4c]**

4b. Rollback of this repair on the test copy

1. Files: unpack writable\app_backups\app-before-payguard-20261015-125912.tar.gz over app\ (or move the five edited files back from the archive) and compare every file with app-before-payguard-20261015-125912.sha256; the two new files are moved to writable_to_delete.

2. Tables: the TEST rows are removed by their listed ids (section 4c), or voucher, v_inv, account_runing, invoice_no, tour_quotations, consignee, driver_allocation and tpd_audit are restored by `RENAME TABLE` from their zz_bak_20261015_1259_ copies; pg_auth_claim and pg_op are dropped. Counts and checksums must then equal the copies.
3. Nothing on development or live is involved; there is nothing to roll back there.

4c. Test evidence from the test copy

Status at 13:24 UTC, 15 October: the tests are prepared but NOT RUN. Every route on the test copy requires a signed-in user, the TEST passwords are in no project fixture, and the sign-in page is still showing in the browser pane; the tests start the moment you sign in as TestAccounts and say so (blocker B-1). Nothing below is claimed as observed until then. **[Observed: sign-in page at 13:24 UTC]**

TEST records created in the test database for these tests (all labelled TEST; listed for removal under 4b):

Record	Ids	Purpose
Vouchers TEST-GUARD-1 to 6 (sup = 0, Bank Transfer, account 6-9-23-11013, Aflag 0), modelled on 7134's structure	voucher ID 8032–8037, Vnum 8021–8026, amounts 1,111.11 … 6,666.66; one v_inv line each (ID 8253–8258)	one voucher per test case below
Voucher TEST-GUARD-TA1 (transport advance, authorised, Aflag 1, 10,000.00, driver 6-9-11-99901, tour 990001) with its ledger journal	voucher ID 8038, Vnum 8027; v_inv ID 8259; account_runing journal 28990 (two legs)	the V5 transfer tests
Counters set to match	invoice_no: V_Num 8027, acc_id 28990	no collision with later creations
Baseline before tests	account_runing 11,392 rows; voucher 7,982; v_inv 8,185; tpd_audit 9,719	the after-counts are compared with these

The test set, with the result each must show (run from the signed-in page, each request's status, message and timing recorded; counts and the audit rows read after each case):

#	Case	Route	Must show
G-1	normal authorisation of TEST-GUARD-1	V1	200; one journal (2 legs) under 8021; Aflag 1; claim “posted”; audit auth_claimed + auth_posted
G-2	double click: two posts fired without waiting, TEST-GUARD-2	V1	one 200 and one 409 with the voucher named; one journal only
G-3	retry after success: TEST-GUARD-1 again	V1	409 “already authorised”; journal count unchanged
G-4	two tabs: the same post from two tabs at once, TEST-GUARD-3	V1	one 200, one 409
G-5	two users at the same moment (or two tabs if only one login is available), TEST-GUARD-4, the first delayed 4 s inside the guarded section	V1	the first 200 after ≈ 4 s; the second waits on the lock, then 409; one journal; both audit rows
G-6	posting failure after the ledger rows were written (test hook), TEST-GUARD-5	V1	500 “failed and was undone”; 0 ledger rows under 8025; Aflag back to 0; claim removed; audit auth_failed_undone naming the deleted journal id; counts equal to before the attempt
G-7	retry after the undone failure, TEST-GUARD-5	V1	200; one journal; the earlier journal id not reused (counter gap recorded)
G-8	reject TEST-GUARD-6, then try to authorise it	V1	reject 200; authorise refused (301/409); no ledger rows
G-9	bulk hotel accept	V4	403 with the withdrawal message; audit row; no count changes
G-10	V5 transfer 3,000 from 99901 to 99902 on TA1 with an operation key	V5	200; one new journal; TA1 amount 7,000; a new authorised voucher for 3,000; audit
G-11	the same operation key again	V5	409 “already submitted”; nothing changes
G-12	transfer above the balance; same driver; a voucher not authorised (TEST-GUARD-6)	V5	409 / 400 / 409 with the reason; nothing changes

#	Case	Route	Must show
G-13	V5 failure after the lines were written (test hook)	V5	500 “undone”; journal, line, voucher and amount restored; counts equal to before
G-14	the Tour Payment Desk’s Paid for a desk voucher	V2	covered by the code path (the desk calls V1’s update() internally); a desk fixture is not in this set — recorded as a limitation unless you want it added

Evidence to be kept for each case: request fields (no credentials), HTTP status, message, sent and received times, the counts and the audit rows read through phpMyAdmin on the test database, and the ledger rows under each TEST voucher number. Where a case cannot be run (two real users), the limitation is written here, not papered over.

5. Test acceptance criteria for the whole workflow

Every test uses controlled TEST records with a written expected result; the existing data is never the expected result. Every run starts from the verified copies (section 2) and ends with the TEST rows listed and the non-TEST tables compared with their copies (count and CHECKSUM TABLE equal). A test passes only when every line of its “must” column holds. **[Proposal]**

Area	Case	Must
Normal processing	request → approve → voucher → authorise → finalize → complete, one TEST obligation	one request row, one voucher, one journal (two legs, balanced, Recpt_No = voucher number), one finalized flag with one bank date, status 2 at the end; the report for the tour shows the one amount
Double click	the same button twice within a second, on authorise, approve, finalize, complete, create	exactly one effect; the second answers with a refusal message naming the voucher or request; one audit row per outcome
Two tabs	the same action from two tabs of one user	as double click

Area	Case	Must
Two users	the same action from two sessions of two users at the same moment (one request delayed inside the guarded section)	one effect; the other refused after the first finishes; both audit rows carry the right user
Retry	the same action again after success, after refusal, after an undone failure	after success: refused; after refusal: refused again; after an undone failure: succeeds once
Interrupted processing	a failure injected after the first write of each multi-write step (ledger rows, status, counter, lines)	tables equal to the state before the attempt (counts and checksums), status unchanged, the counter not consumed or its gap recorded, an audit row “failed, undone”, a message that says nothing was posted
Partial payments	a request paid in two vouchers; a voucher that overpays; a balance request	allocations sum to the request; overpayment refused; completion only when the sum covers the request
Cancellations	cancel at each status (1, 4, 6, 2) and a cancelled voucher	allowed only at 1 with no voucher; all others refused with the status named; a cancelled voucher cannot be authorised or finalized
Alternative routes	every route in the inventory (A2 section 4a) that changes a payment status or posts: V1–V5, P1–P7, Q1–Q2, X1, B1/A1, creation routes, manual tools	each either applies the same engine or is withdrawn with a 403 and an audit row; none can change a status out of sequence
Numbering	20 creations fired together	20 distinct voucher numbers; each line attached to its own voucher; no gap larger than the recorded undone attempts
Permissions	each payment action by each TEST role, and by a user with no role	allowed only for the role named in the plan; refused with a message otherwise; refusal audited; a forged cross-site POST without the token refused

Area	Case	Must
Audit	after every test above	one row per attempt with action, user, voucher or request, outcome and reason; no row can be updated or deleted by the application (trigger refuses)
Reports agree with records	after the suite	every report total equals the sum of the stored records it claims to summarise; no figure comes from a flag
Storage engine	the integrity cases above repeated after the MyISAM-or-InnoDB decision (R-10)	identical results; under InnoDB, additionally no undo code path is needed to reach the same state

Evidence of each run: the request and response (status, message) per case, the before and after counts and checksums, the audit rows, and the TEST record ids, saved in the evidence folder under a dated sub-folder by the capture route you approve.

6. Preparing the clean database for fresh entry (structure and configuration only)

After validation, not before. A separate, empty database is prepared only when sections 3–5 are complete on the test copy and you have approved the repaired code. It carries structure and approved configuration, and nothing else. **[Proposal]**

Carried into the clean database	Not carried
the schema of every table, with the repaired keys and indexes (unique journal id, unique voucher number, business keys, append-only audit triggers) and the storage engine you decide under R-10	any transaction: no vouchers, lines, ledger rows, requests, payments, bank statements or audit rows from the old data
configuration tables you approve: chart of accounts and ledger codes, bank accounts and their mappings, payment types, menus and privileges as you assign them	unverified balances: no opening balance is entered by the system; you and Ravi supply the opening balances separately, and they are entered as dated journals under the normal, guarded routes

Carried into the clean database	Not carried
master data you confirm: suppliers, hotels, drivers, agents (each list reviewed by you before load; nothing merged or deduplicated automatically)	logins and passwords: accounts are created by you; no TEST login is carried
the counters set to the starting values you choose	historical reports; the old database stays intact and readable beside it

Starting date. Not assumed. The year-end on 31 March and a 1 April start are noted as your intention; the date, the master data and the opening balances are confirmed by you separately, and nothing is imported or reset until then.

Method, when approved. The structure is produced from the repaired test copy by the application's own schema definitions (the installer libraries and app\Database\Sql, extended for the repairs), loaded into a new database name of your choosing on the same MySQL server, and verified: object counts, table engines, keys and triggers equal to the test copy's; row counts zero except the configuration and master tables you approved; a dry run of the full test suite (section 5) on the clean database before any staff use. The old database and its files are preserved unchanged throughout.

7. Genuine blockers and the approvals needed next

Blockers (nothing else stops the work)

#	Blocker	What resolves it
B-1	The TEST sign-in on the test copy: the TEST passwords are in no project fixture, so only you can sign in; without it no write test runs	you sign in as TestAccounts in the browser pane (and, for the two-user case, a second Test* login in your own Chrome)
B-2	The standing of avenir_rv under the structural restriction (section 2, item 1)	one line from you
B-3	The storage-engine decision R-10 (keep MyISAM with lock-claim-undo everywhere, or convert the eight tables to InnoDB) — it fixes the shape of R-2, R-4, R-5 and R-7	your decision, after the InnoDB rehearsal of r3 section 9 if you want it

#	Blocker	What resolves it
B-4	The screen and column behind “paid items reappear” and “invoices appear two or three times” (Final package 7d): the repairs R-2/R-3 cover every return point on record, but the specific report you saw cannot be reproduced until it is named	the screen's name as you see it, the column, one example reference
B-5	A database backup outside the MySQL server for the test copy (section 2, item 2) — not needed for the tests, needed before the clean-database step	S1–S3 of the Final package (MySQL file output) or an IT-run dump

Approvals needed next, in order

#	Approval	Unlocks
A-1	The guard (R-1) as built, on the evidence of 4c, as the pattern for every other posting route	R-2 to R-7 built the same way
A-2	R-10: MyISAM with lock-claim-undo, or InnoDB conversion first	the design of R-2, R-4, R-5, R-7
A-3	Build R-2/R-3/R-6 (the status engine) on the test copy, with its TEST fixture	the next build
A-4	The capture route for test evidence into the evidence folder (Final package section 3: C1 after S1–S3, or C2)	durable test evidence
A-5	Installation of tf-payguard-r1 on development, after a verified backup, at a time you choose — **not asked for now**; live is not covered by any of this	nothing until you ask
A-6	The clean database (section 6): name, starting date, master data, opening balances, logins	the last step

Not needed from you now: staff assignments (yours, later); Ravi's involvement (the 7134 pack stays prepared in the Final package, unsent); any correction to an old record.